

RKLLM 模型转换完全指南

从 HuggingFace 模型 / 微调产物 到 .rkllm 部署格式

RK3576 / RK3588 NPU 部署系列

配套脚本: `scripts/export.py` `scripts/export_qwen3.py`

官方参考: github.com/airockchip/rknn-llm

2026-08-11

目录

- 1 为什么要转换：HF 模型在 NPU 上跑不起来
- 2 转换的本质：RKLLM-Toolkit 到底做了什么
- 3 官方三阶段 API：load → build → export
 - 3.1 加载模型 load_huggingface
 - 3.2 构建模型 build
 - 3.3 导出模型 export_rkllm
- 4 量化深度解析：W8A8 / W4A16 / GRQ / 校准数据
- 5 目标平台与 NPU 配置：target_platform / num_npu_core / max_context
- 6 微调模型转换专题（重点）
 - 6.1 LoRA 合并 vs model_lora 参数
 - 6.2 自定义词表检查
 - 6.3 转换前验证清单
- 7 常见错误与排查
- 8 你目录里的脚本逐行解析
- 9 参考资源

1. 为什么要转换：HF 模型在 NPU 上跑不起来

HuggingFace 上下载模型（Qwen3-4B、DeepSeek-R1-Distill 等）是一个 PyTorch 权重仓库：config.json + safetensors/pytorch_model.bin + tokenizer。它只能在 PC 上用 PyTorch 推理。而 RK3576/RK3588 板子上只有 8~16GB 内存和 NPU，没有 NVIDIA 显卡，没有完整的 PyTorch 运行时，直接跑会遇到四个问题：

障碍	说明	后果
内存	FP16 的 4B 模型权重约 8GB，激活值还要额外内存；W8A8 量化后约 4GB	放不下 / 无余量跑长上下文
精度	FP16 对 NPU 不是最优精度，NPU 擅长 INT8 矩阵乘	算得慢、功耗高
算子	PyTorch 里几百种算子，NPU 硬件只实现了其中一部分（GELU、RMSNorm、RoPE 等需映射/融合）	无法直接执行
运行时	板端没有 CUDA/PyTorch，只有 C 库	需要 C/C++ API 驱动格式

RKLLM 软件栈就是解决这四件事的官方方案，由三部分组成：

- RKLLM-Toolkit —— PC 端工具（Python 包 rkllm.api），把 HF/GGUF 模型转换成 .rkllm 格式（本指南主角）
- RKLLM Runtime —— 板端 C/C++ 库，加载 .rkllm 并调用 NPU 推理
- RKNPU 内核驱动 —— 已开源，集成在 Rockchip 内核里

注意：转换在 PC 上进行（x86 主机，建议 32GB 以上内存，有 NVIDIA 显卡更快），板子上不需要装 rkllm-toolkit，只需要 Runtime 和模型文件。

2. 转换的本质：RKLLM-Toolkit 到底做了什么

一句话：把【PyTorch 权重】翻译成【NPU 能直接执行的程序 + 权重二进制】。具体分四步：

步骤	做了什么	对应代码
① 结构解析	读 config.json，按模型架构（Qwen3 = Llama 类结构）重建计算图，把 Attention/RoPE/RMSNorm/GELU 等算子映射到 NPU 支持的算子集合	load_huggingface
② 权重重排	把 HF 权重张量按 NPU 要求重新排布（reshape/permute/分块），并合并为 NPU 友好的布局	build 内部
③ 量化	把 FP32/FP16 权重（和激活值）压缩为 INT8/INT4，附上 scale/zero-point 参数，压缩体积、提速（详见第 4 章）	build(do_quantization=True)
④ 算子融合	把相邻算子融合（如 QKV 投影合并、激活函数并入前一层），减少 NPU 调用次数	build(optimization_level)

关键认知：转换工具【不学习任何东西】。它不训练、不微调，只负责把已有的权重重新打包。所以——微调过的模型只要能正常加载进 PyTorch，就能被转换，转换结果会完整保留微调学到的能力。

官方支持两种输入格式：

- HuggingFace 格式目录：config.json + safetensors(.bin) + tokenizer，用 load_huggingface() 加载

- GGUF 格式文件 (llama.cpp 生态) : 用 `load_gguf()` 加载
 - 自定义架构 (非标准模型) : 通过 `custom_config` + `config_custom.json` 描述网络结构, 仓库 `examples/custom_demo` 里有例子, 一般用不上

3. 官方三阶段 API：load → build → export

官方转换流程固定三步，缺一不可：

```
from rkllm.api import RKLLM
llm = RKLLM()
# ① 加载：把 HF 模型读进内存，构建计算图
llm.load_huggingface(model=modelpath, model_lora=None,
                     device='cpu', dtype='float16')
# ② 构建：量化 + 优化 + 算子融合（最耗时，1.5B 约几十分钟，4B 约 1~2 小时）
llm.build(do_quantization=True, optimization_level=1,
          quantized_dtype='w8a8', target_platform='rk3588',
          num_npu_core=3, dataset=dataset, max_context=4096)
# ③ 导出：写出 .rkllm 文件，拷到板子上
llm.export_rkllm('./model.rkllm')
```

3.1 加载模型 load_huggingface

参数	取值	作用 / 建议
model	HF 模型目录绝对路径	目录里必须有 config.json；权重用 safetensors 或 .bin 均可
model_lora	None 或 LoRA 目录路径	None=用模型自带权重；指定路径=加载 LoRA adapter 并合并（见 6.1）
device	'cpu' / 'cuda'	cpu 保险但慢（4B 需 ~16GB 内存）；cuda 快得多，有 N 卡就选 cuda
dtype	'float32' / 'float16' / 'bfloat16'	加载精度。float16 省一半内存、速度更快，精度略降；建议默认 float16
custom_config	None / 自定义配置对象	非标准架构才需要，标准模型保持 None
load_weight	True / False	是否加载权重。一般保持 True

报 'Load model failed!' 时优先检查：model 路径是否有 config.json；路径是否绝对路径；内存是否足够（dtype 改成 float16 或加 --cuda 换显卡）；模型架构是否官方支持列表内。

3.2 构建模型 build

参数	取值	作用 / 建议
do_quantization	True / False	True=量化（体积小速度快）；False=保留高精度（fp16），仅调试用
quantized_dtype	w8a8 / w4a16 / w8a8_gx / w4a16_gx	量化方案，见第 4 章
quantized_algorithm	normal / grq / gptq	量化算法，与 dtype 配套，见 4.2
optimization_level	0 / 1 / 2	0=不优化（调试）；1=推荐，平衡速度与精度；2=更激进优化（追求速度）
target_platform	'rk3588' / 'rk3576' 等	目标芯片，必须和板子一致，RK3576 用 'rk3576'
num_npu_core	1 ~ 3	NPU 核数。RK3588 有 3 核；RK3576 有 2 核，填 2

dataset	校准 json 路径	do_quantization=True 时必须提供，用于统计激活值范围（见 4.3）
extra_qparams	None / ExtraQParams 对象	进阶：控制嵌入层/输出层/个别层是否量化；默认 None 即可
hybrid_rate	0 ~ 1	混合量化比例：多少比例的层保持高精度不量化。0=全量化（推荐先跑 0）；精度不够时再调大，代价是体积变大
max_context	整数（token 数）	支持的最大上下文长度。按实际需要设，太大浪费内存

build 是整条链路里最容易失败、最耗时的环节。失败时先确认：dataset 路径存在且格式正确；target_platform 与芯片一致；内存足够（4B 模型 build 峰值可到 32GB+）。

3.3 导出模型 export_rkllm

把构建结果写成一个 .rkllm 文件。文件命名建议带上信息：

```
llm.export_rkllm(f'../model/{模型名}_{量化方案}_{平台}.rkllm')  
# 例：Qwen3-4B-Instruct_W8A8_RK3588.rkllm
```

- 生成的 .rkllm 是【自包含】的：权重 + 计算图 + 量化参数 + tokenizer 表全部打包在一个文件里
- 拷贝到板子上，RKLLM Runtime 的 C API（rkllm_init / rkllm_run / rkllm_release）直接加载
- 注意板端内存：W8A8 4B 模型 + max_context 4096 运行时大约占 5~6GB

4. 量化深度解析

4.1 为什么必须量化

LLM 推理瓶颈是权重搬运：每生成一个 token 都要把全部权重从 DDR 读一遍。FP16 权重 2 字节/个，INT8 只要 1 字节，INT4 只要 0.5 字节——权重小一半/四倍，内存占用和每个 token 读取带宽压力同比例下降，速度近似翻倍。这就是 RKLLM 部署必做量化的原因。

4.2 量化方案怎么选

方案	量化对象	体积（4B 模型）	推荐算法	适用场景
w8a8	权重 8bit + 激活 8bit	~4GB	normal	首选！速度/精度均衡，16GB 板子轻松装下
w4a16	权重 4bit + 激活 16bit	~2GB	grq	追求最小体积，精度下降明显，需实测
w8a8_gx	权重 8bit，按分组量化	~4GB	normal	精度比 w8a8 更高，体积略增
w4a16_gx	权重 4bit 分组量化	~2GB	grq	w4a16 的精度改良版
gptq-int8	权重 8bit（GPTQ 算法）	~4GB	gptq	对精度敏感的场景

分组（_gx）的意思：不是整个张量共用一个 scale，而是把权重按 group 分组（如每 64/128/256 个元素一组），每组独立统计 min/max 算 scale。组越小精度越高，但需要的存储和计算也越多。

工程建议：第一版直接用 w8a8 + normal + optimization_level=1。跑通全链路后再尝试其他组合对比精度。

4.3 校准数据集 dataset —— 最容易忽视的关键输入

做激活量化（w8a8）时，NPU 需要知道激活值的实际分布范围才能确定 scale。做法：把一批真实输入喂给模型跑一遍 forward，统计每层激活值的 min/max/分布，这叫【校准（calibration）】。这个校准用的数据就是 dataset 参数指向的 json。

JSON 格式（官方规定，注意必须是列表，每个元素含 input 和 target）：

```
[{"input": "Human: 你好! \nAssistant: ", "target": "你好! 我是助手"}, ...]
```

你目录里的 scripts/data_quant.json 就是这个文件（19 条中英混合样本）。选择要点：

- 条数：建议 32~128 条。你现在 19 条偏少，校准出的激活范围可能不准，可酌情加到 40+
- 内容：要和【实际使用场景】分布接近——如果板子上跑 OCR/中文问答，校准样本就用中文问答，而不是英文小说
- input 要带对话模板前缀（Human: ... Assistant: 或模型要求的 chat template），与推理时一致
- target 是参考答案，校准主要用 input，target 保证格式合法即可（框架用它拼完整序列）

校准数据不参与学习，只统计数值范围。校准数据质量不好 → 激活 scale 不准 → 推理精度下降，这是'模型转换出来变笨了'最常见的原因。

5. 目标平台与 NPU 配置

参数	推荐值	说明
target_platform	rk3576 / rk3588	必须与板子一致。转错平台文件无法加载
num_npu_core	rk3588=3, rk3576=2	用满核数，注意板子散热；只填 1 可省功耗
max_context	4096 / 8192	按需设置。4B w8a8 + 8192 上下文在 16GB 板子上已偏紧
hybrid_rate	0	先全量化；精度不行再逐步调大（如 0.2）
optimization_level	1	0 调试 / 1 推荐 / 2 速度优先

RK3576 与 RK3588 的区别：RK3588 是 3 核 NPU（6 TOPS），RK3576 是 2 核 NPU（6 TOPS 同代但核数不同）。同样的模型转两个平台要分别 build 两次（target_platform 不同）。

6. 微调模型转换专题（重点）

你的场景：用 LoRA 微调（如 scripts/finetune_classifier.py 微调 Qwen2-VL-2B OCR），然后希望把微调成果部署到板子上。转换本身不关心模型是否微调过，但三个细节决定成败：LoRA 怎么进转换、词表变没变、转换前先验证。

6.1 LoRA 合并 vs model_lora 参数（二选一，必须选一个）

LoRA 微调产出的是【主模型权重 + 少量低秩 adapter 增量】。转换器要么拿到完整权重，要么拿到主权重 + adapter 路径，否则转换结果等于没微调。

方式 A：微调后合并再保存（推荐，最稳）

```
from peft import PeftModel
model = PeftModel.from_pretrained(base_model, './lora_output')
merged = model.merge_and_unload()          # 把 adapter 并入主模型
merged.save_pretrained('./model_merged')   # 标准 HF 格式，直接喂转换
tokenizer.save_pretrained('./model_merged')
```

方式 B：不合并，转换时用 model_lora 参数

```
llm.load_huggingface(model=base_path,
                     model_lora='./lora_output', # 官方自动合并
                     device='cpu', dtype='float16')
```

注意（最常犯的错）：微调完只保存了 adapter，转换时 model_lora 又不传（或传错路径）——转换照常成功，但出来的模型和没微调完全一样，还不好排查。确认方法：转换日志里应出现加载 lora 的信息，或对比转换前后权重文件体积。

6.2 自定义词表检查

如果微调时给 tokenizer 加过新 token（如 <ocr>、<image> 等特殊符号）：

- tokenizer 必须重新保存，且 config.json 的 vocab_size 与 tokenizer 实际大小一致
- 新增 token 的 embedding 是随机初始化的，转换/量化会把随机初始化的权重也打包进去，建议微调时用 add_special_tokens 后 re-init 或训练充分覆盖新 token
- RKLLM 会按 config.json 读词表，vocab_size 对不上会报错或生成乱码

6.3 转换前验证清单（转换前必做）

#	检查项	怎么查
1	微调产物能加载、能推理	PC 上 transformers 加载模型，跑一条真实输入，确认输出是微调后的效果
2	LoRA 已合并（或 model_lora 已配置）	见 6.1，转换日志确认 lora 加载
3	词表/特殊 token 一致	tokenizer.vocab_size 与 config.vocab_size 相等
4	校准数据符合目标场景	dataset 样本与板子实际使用场景分布一致，40+ 条
5	内存足够	build 4B 模型建议 ≥32GB 内存或 cuda 显卡
6	目标平台写对	rk3576 板子填 rk3576，rk3588 填 rk3588

7	板端验证	部署后跑同一句测试输入，与 PC 输出对比（允许量化引起的轻微差异）
---	------	------------------------------------

7. 常见错误与排查

报错 / 现象	原因	解决
Load model failed	路径错 / 内存不足 / 架构不支持	检查 config.json; dtype 改 float16; 换 cuda; 对照官方支持模型列表
Build model failed	dataset 路径或格式错 / 内存爆 / 平台写错	dataset 必须是 [{"input":..., "target":...}] 列表; 扩容内存; target_platform 与芯片一致
转换成功但板端加载失败	target_platform 与板子不一致 / Runtime 版本太旧	确认平台; 升级 RKLLM Runtime 到与 Toolkit 匹配版本
输出乱码 / 中文变问号	tokenizer 词表缺失或 vocab_size 不一致	重新保存完整 tokenizer; 检查特殊 token; 用官方 chat template 拼 input
转换后效果明显变差	校准数据不合格 / 量化过狠	校准样本贴合场景并加到 40+; 尝试 w8a8_gx 或 hybrid_rate=0.2; 必要时 w4a16 换回 w8a8
推理速度慢 / 内存不够	max_context 太大 / NPU 核没用满	按需缩小 max_context; num_npu_core 调到芯片最大值
Python 3.12 装不上 / 报错	官方 wheel 是 cp38/cp39 构建	用 Python 3.9~3.12 (README 注明 3.12 需 export BUILD_CUDA_EXT=0); 或直接用官方 Docker 镜像

8. 你目录里的脚本逐行解析

8.1 scripts/export.py (DeepSeek-R1-Distill-Qwen-1.5B)

```
# 加载阶段
llm.load_huggingface(model=modelpath, model_lora=None, device='cpu',
                      dtype='float16', custom_config=None, load_weight=True)
# device='cpu': 没 GPU 就用 CPU 转 (1.5B 无压力)
# dtype='float16': 省内存, CPU 加载 1.5B 足够快
# model_lora=None: 当前用原生权重, 微调后这里要接 LoRA (见 6.1)

# 构建阶段
dataset = './data_quant.json'          # 19 条校准样本 (建议加到 40+)
target_platform = 'RK3588'             # ← 注意: 你的板子是 RK3576 就改 'rk3576'
optimization_level = 1                 # 推荐档位
quantized_dtype = 'W8A8'               # 首选方案
quantized_algorithm = 'normal'         # w8a8 配套 normal
num_npu_core = 3                      # RK3588=3; RK3576=2
llm.build(do_quantization=True, optimization_level=1,
          quantized_dtype='W8A8', quantized_algorithm='normal',
          target_platform='RK3588', num_npu_core=3,
          extra_qparams=None, dataset=dataset, hybrid_rate=0,
          max_context=4096)
```

注意: 脚本写死 RK3588/3 核。如果部署目标其实是 RK3576, target_platform 要改成 rk3576、num_npu_core 改成 2, 否则 .rkllm 在 RK3576 上加载失败。

8.2 scripts/export_qwen3.py (Qwen3-4B-Instruct)

```
# 与 export.py 几乎相同, 区别:
llm.load_huggingface(model=modelpath, model_lora=None, device='cpu',
                      dtype='float32', ...) # 4B 用 float32 更稳, 但内存需求大
# — 官方注释也提醒: 4B 模型 CPU 模式约需 16GB RAM
llm.build(..., max_context=8192)          # 上下文比 1.5B 版开得更大
# 注意: w8a8 + 8192 上下文在 16GB 板上运行时会偏紧, 实测后再决定
```

8.3 微调脚本怎么配合转换 (finetune_classifier.py → 转换)

你的 finetune_classifier.py 目前保存 LoRA adapter (output 目录)。转换前补上合并步骤, 或转换时指定 model_lora:

```
from peft import PeftModel, AutoPeftModelForCausalLM
# 方式 A (推荐): 合并成完整模型
merged = model.merge_and_unload()
merged.save_pretrained('./model/Qwen2-VL-2B-OCR-merged')
tokenizer.save_pretrained('./model/Qwen2-VL-2B-OCR-merged')
# 然后 load_huggingface(model='./model/Qwen2-VL-2B-OCR-merged')

# 方式 B: 转换时挂 adapter
llm.load_huggingface(model='./model/Qwen2-VL-2B-Instruct',
                      model_lora='./model/Qwen2-VL-2B-OCR/lora_adapter')
```

9. 参考资料

资源	位置
官方仓库 (源码/文档/示例)	github.com/airockchip/rknn-llm (国内镜像 gitee.com/airockchip/rknn-llm)
官方 SDK 完整包 (含中文手册 PDF)	RKLLM_SDK: console.zbox.filez.com/l/RJJdMB 取件码 rkllm

官方用户手册	仓库 doc/Rockchip_RKLLM_SDK_CN_1.3.0.pdf
已转换模型库	rkllm_model_zoo: console.box.lenovo.com/l/10tXb8 取件码 rkllm
支持模型列表	官方 README 'Support Models' (Qwen3/Qwen2.5/DeepSeek-R1-Distill/多模态等)

核心结论回顾：官方转换不训练、不微调，只做【结构解析算子融合】；微调成果通过两种方式进入转换：合并后的完整 adapter；转换质量的最大变量是校准数据质量与平台参数正确性。

→ 权重重排 HF 模型，或 → 量化 model_lora → 参数挂